ΓΛΩΣΣΑ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΥ C++

ΔΕΙΚΤΕΣ ΜΝΗΜΗΣ - Nodes

ΔΥΝΑΜΙΚΗ ΔΙΑΧΕΙΡΙΣΗ ΜΝΗΜΗΣ

ΜΕ ΔΕΙΚΤΕΣ ΜΝΗΜΗΣ

για την αναφορά σε δομές δεδομένων (πίνακες)

Το Όνομα ενός Πίνακα είναι Δείκτης

- Οι δείκτες είναι πολύ χρήσιμοι σε συνδυασμό με τους πίνακες!
- Αυτό συμβαίνει διότι εξ' ορισμού <u>το όνομα κάθε πίνακα είναι ένας δείκτης</u> με τύπο δεδομένων τον τύπο δεδομένων του πίνακα.
- Ωστόσο είναι ένας δείκτης που δεν μπορούμε να αλλάξουμε την τιμή του.
  - Μπορούμε όμως να ορίσουμε έναν δείκτη να δείχνει στην αρχή του πίνακα με τον εξής τρόπο:

```
int pinakas[100];
int *ptr;
ptr=pinakas;
```

```
// Βάλε τον δείκτη ptr να δείχνει
στον πρώτο ακέραιο αριθμό του πίνακα.
```

  - μπορούμε να βάλουμε τον δείκτη να δείχνει στο πρώτο στοιχείο του πίνακα με την εντολή

```
int pinakas[100];
int *ptr;
ptr=&pinakas[0];
```

```
// Βάλε τον δείκτη ptr να δείχνει
στον πρώτο ακέραιο αριθμό του πίνακα.
```

Αποθήκευση ενός Πίνακα στη Μνήμη

- με την δήλωση: `int pin[4];`
  Δημιουργείται ένας πίνακας 4 διαδοχικών θέσεων στην μνήμη

- Γνωρίζουμε πλέον ότι &pin[0],&pin[1],&pin[2],&pin[3] είναι οι διευθύνσεις μνήμης των 4 μεταβλητών του πίνακα.
- Το επόμενο πρόγραμμα δείχνει ότι όντως οι μεταβλητές αυτές αποθηκεύονται σε διαδοχικές θέσεις μνήμης.

➢ Μεταγλωττίστε, εκτελέστε και μελετήστε το πρόγραμμα:

```
#include <stdio.h>

#define N 4

main()
{
    int pin[N];
    int i;

    for (i=0; i<N; i++)
        printf("\nStoixeio:%d, Diefthinsi Thesis Mnimis: %d",i, &pin[i]);
}
```

τι θα συμβεί ?

➢ Μεταγλωττίστε, εκτελέστε και μελετήστε το πρόγραμμα:

```
#include <stdio.h>

#define N 4

main()
{
    int pin[N];
    int i;

    for (i=0; i<N; i++)
        printf("\nStoixeio:%d, Diefthinsi Thesis Mnimis: %d",i, &pin[i]);
}
```

Θα δείτε ότι και οι "θέσεις μνήμης/διευθύνσεις" είναι διαδοχικές

Η ΙΔΙΑΙΤΕΡΗ ΣΥΜΠΕΡΙΦΟΡΑ

ΤΩΝ ΔΕΙΚΤΩΝ ΜΝΗΜΗΣ

ΣΤΗΝ ΔΙΑΠΕΡΑΣΗ ΤΩΝ ΠΙΝΑΚΩΝ

- Οι δείκτες έχουν μια ιδιαίτερη αριθμητική έτσι ώστε να δουλεύουν σε συνδυασμό με πίνακες:

- Έστω το πρόγραμμα

```
int pinakas[100];
int *ptr;
ptr=pinakas;
```

δείκτης μνήμης

```
// Βάλε τον δείκτη ptr να δείχνει
στον πρώτο ακέραιο αριθμό του πίνακα.
```

```
int pinakas[100];
int *ptr;
ptr=&pinakas[0];
```

γράφεται και έτσι

- η εντολή:

```
ptr++;
```

  - Δεν αυξάνει την τιμή του δείκτη κατά 1!
  - Αλλά αυξάνει την τιμή του δείκτη κατά τόσα bytes όσα είναι και ο τύπος δεδομένων του δείκτη (για ακέραιο: κατά 4!).

- Ενώ η εντολή:

```
ptr--;
```

  - Μειώνει την τιμή του δείκτη κατά τόσα bytes όσα είναι και ο τύπος δεδομένων του δείκτη (για ακέραιο: κατά 4!).

Γράψτε και εκτελέστε το επόμενο πρόγραμμα σε C++, για να μελετηθεί η συμπεριφορά μέσα σε πίνακα, ενός <u>δείκτη μνήμης.</u>

```
#include <stdio.h>

#define N 10

main()
{
   int pin[N];
   int i;
   int *ptr;
   ptr=pin;

   for (i=0; i<N; i++)
   {
      printf("\nStoixeio:%d, Diefthinsi Thesis Mnimis: %d",i, ptr);
   }  ptr++;
}
```

δείκτης μνήμης

Θυμίσου...

```
int pinakas[100];
int *ptr;
ptr=pinakas;
```

δείκτης μνήμης

```
// Βάλε τον δείκτη ptr να δείχνει
```

στον πρώτο ακέραιο αριθμό του πίνακα.

παράδειγμα

```
double pinakas[100];
double *ptr;

ptr=pinakas;
ptr+=2;
```

- Η εντολή `ptr+=2` θα αυξήσει την τιμή της `ptr` κατα 2 x (το μέγεθος του double)
- άρα θα αυξήσει τη διεύθυνση κατά 16, αφού μία double μεταβλητή δεσμεύει 8 bytes.
- Μετά το πέρας της εκτέλεσης αυτών των εντολών ο δείκτης `ptr` θα δείχνει στο στοιχείο `pinakas[2]`

σε αντιπαράθεση με το κλασσικό τρόπο, να προστεθεί στο προηγούμενο πρόγραμμα και να εκτελεστεί ο επόμενος κώδικας σε C++, για να μελετηθεί η συμπεριφορά μέσα σε πίνακα, ενός δείκτη μνήμης, στην διαπέραση πίνακα

Με βάση την αριθμητική δεικτών, η παράσταση:

* (array +2) Μπορεί να εξηγηθεί ως εξής:

- Το (array+2) δίνει την διεύθυνση του 3ου στοιχείου του πίνακα.
- Άρα με το * μπροστά, μας δίνει την τιμή του 3ου στοιχείου του πίνακα.

για παράδειγμα ο βρόχος:

```
for (i=0; i<N; i++)
   printf("%d",array[i]);
```

κώδικας κλασσικός

μπορεί να γραφεί ισοδύναμα:

```
for (i=0; i<N; i++)
   printf("%d",*(array+i));
```

κώδικας με δείκτη μνήμης